Query.Farm

SWIPE →

DUCKDB / REFERENCE COLLECTION

32 DuckDB extensions. One page each.

What each extension is for, and a real query that shows it working.



WHAT'S INSIDE

Six kinds of extension

Connectors

9

- ADBC Scanner
- Airport
- HTTP Client
- HTTP Server
- Radio
- Redis
- Shellfs
- Tributary
- VGI

Transformation

10

- Crypto
- Hashfuncs
- Inflector
- JSONata
- MiniJinja
- QuickJS
- RapidFuzz
- Rhai
- Tera
- TSID

Analytics

3

- A5
- Apache DataSketches
- Stochastic

Performance

4

- Bitfilters
- GeoSilo
- Lindel
- Marisa

Data quality

1

- JSON Schema

Developer tools

5

- Cronjob
- Events
- Fuzzycomplete
- Textplot
- WebMacro

ADBC Scanner



Database drivers. Arrow batches. Ordinary SQL.

ADBC driver → Arrow batches → DuckDB SQL

Connect DuckDB to a database through an Arrow Database Connectivity driver. Use attached catalogs for joins and discovery, send backend SQL explicitly, or bulk-ingest a DuckDB query through the driver's Arrow interface.

Query through a driver

REQUIRES SETUP

```
ATTACH ':memory:' AS db (
  TYPE adbc, driver 'sqlite'
);
SELECT * FROM adbc_scan(
  'db', 'SELECT ? AS id',
  params := row(42),
  columns := {'id': 'BIGINT'}
);
```

```
INSTALL adbc_scanner FROM community; LOAD adbc_scanner
```

Airport



Bring Arrow Flight services into your SQL.

Flight server → Arrow stream → DuckDB catalog

Attach an Arrow Flight service as a DuckDB catalog, then query its tables alongside local data. Discover available flights, fetch a descriptor directly, and invoke server-defined actions when a full catalog is unnecessary.

Query an attached catalog

REQUIRES SETUP

```
ATTACH 'grpc://localhost:8815' AS remote
  (TYPE airport);
SHOW TABLES FROM remote.public;
SELECT user_id, count(*) AS n
FROM remote.public.activity
GROUP BY user_id
ORDER BY n DESC
LIMIT 10;
DETACH remote;
```

```
INSTALL airport FROM community; LOAD airport
```

HTTP Client



Small HTTP requests, directly from DuckDB.

HTTP request → JSON envelope → SQL projection

Fetch HTTP resources or submit JSON and form bodies from a SQL expression. Responses arrive as a JSON envelope containing status, reason and body, ready to inspect or save in a local table.

Fetch once, inspect locally

REQUIRES SETUP

```
CREATE TEMP TABLE response AS
SELECT http_get('https://example.com/') AS r;
SELECT (r->>'status')::INTEGER AS status,
       r->>'reason' AS reason,
       r->>'body' AS body
FROM response;
```

```
INSTALL http_client FROM community; LOAD http_client
```

HTTP Server



Serve a DuckDB connection through HTTP.

HTTP client → SQL request → DuckDB result

Start an HTTP listener inside a DuckDB process and query it from an HTTP client. Send SQL in a POST body, receive JSON results, and keep the process running for the lifetime of the service.

Start a local listener

REQUIRES SETUP

```
SELECT httpserve_start(
  'localhost', 9999, 'replace-this-key'
);
```

```
INSTALL httpserver FROM community; LOAD httpserver
```

Radio



Live channel messages meet relational queries.

Live channel → Memory buffer → DuckDB rows

Subscribe to WebSocket or Redis Pub/Sub endpoints, query buffered messages, and queue replies from DuckDB. The subscription URL identifies the feed throughout its lifecycle, while background I/O keeps network activity separate from buffer scans.

Subscribe, wait and inspect

REQUIRES SETUP

```
SET VARIABLE feed = 'ws://localhost:8765/';
CALL radio_subscribe(getvariable('feed'));
CALL radio_listen(true, INTERVAL '5 seconds');
SELECT receive_time, channel,
       decode(message) AS text
FROM radio_received_messages()
WHERE subscription_url = getvariable('feed')
ORDER BY receive_time DESC
LIMIT 20;
```

```
INSTALL radio FROM community; LOAD radio
```

Redis



Read hot data. Analyze it with SQL.

Redis keys → SQL functions → Local analysis

Query Redis strings, hashes and lists from DuckDB, or snapshot operational data into local tables for analysis. Every Redis call selects a named DuckDB secret, keeping connection details separate from the data-access expression.

Snapshot matching hashes

REQUIRES SETUP

```
CREATE TABLE users_snapshot AS
SELECT key, field, value
FROM redis_hscan_over_scan(
  'user:*', '*', 100, 'quickref_redis'
);
SELECT key, count(*) AS fields
FROM users_snapshot
GROUP BY key;
```

```
INSTALL redis FROM community; LOAD redis
```

Shellfs



Read and write shell pipelines through file paths.

Local command → Streaming pipe → DuckDB rows

Use a shell command as a DuckDB file source or sink. A trailing pipe reads command output; a leading pipe sends COPY output to a command. Useful for local tools that already produce or consume CSV and JSON.

Query a command's CSV output

```
SELECT sum(n) AS total
FROM read_csv(
  'printf "n\n1\n2\n3\n" |',
  header = true
);
-- 6
```

```
INSTALL shellfs FROM community; LOAD shellfs
```

Tributary



Query Kafka topics from DuckDB.

Kafka topic → Message rows → SQL analysis

Read Kafka topic messages as rows with topic, partition, offset and BLOB payload columns. Discover cluster metadata, decode application payloads, and materialize a local table for joins and repeatable downstream analysis.

Snapshot a partition range

REQUIRES SETUP

```
CREATE TABLE topic_snapshot AS
SELECT partition, "offset", message
FROM tributary_scan_topic('events',
  "bootstrap.servers" := 'localhost:9092',
  "group.id" := 'quick-reference',
  "enable.auto.commit" := 'false'
)
WHERE partition = 0
  AND "offset" BETWEEN 0 AND 99;
SELECT "offset",
       decode(message)::JSON->>'user_id' AS id
FROM topic_snapshot;
```

```
INSTALL tributary FROM community; LOAD tributary
```

VGI



Attach a worker. Query its capabilities.

Worker code → Arrow IPC → DuckDB SQL

Vector Gateway Interface connects DuckDB to workers written in other languages. A worker advertises catalogs, tables and functions; attach a catalog and query it alongside local data over a subprocess or HTTP connection.

Attach and query

REQUIRES SETUP

```
ATTACH 'analytics' AS app (
  TYPE vgi,
  LOCATION 'https://worker.example.com/vgi/'
);
SHOW TABLES FROM app.main;
SELECT * FROM app.main.events
LIMIT 10;
DETACH app;
```

```
INSTALL vgi FROM community; LOAD vgi
```

Crypto



Digests, message authentication & random bytes.

SQL values → Hash / HMAC → BLOB or hex

Compute cryptographic digests and HMACs directly from SQL values. Use secure random bytes for tokens and salts, or aggregate values in a specified order for reproducible checksums.

Bytes & ordered aggregates

```
SELECT octet_length(crypto_random_bytes(32));

SELECT to_hex(crypto_hash_agg(
  'sha2-256', s ORDER BY s
)) AS digest
FROM (VALUES ('A'), ('B')) AS t(s);
```

```
INSTALL crypto FROM community; LOAD crypto
```

Hashfuncs

Fast fingerprints, buckets & cache keys.

Canonical input → Named hash → Bucket / key

Hash SQL values with named non-cryptographic algorithms. Choose a stable representation and seed for repeatable partitioning, probabilistic-filter inputs and compact fingerprints.

Choose the output width

```
SELECT typeof(xxh3_64('hello')) AS small,
       typeof(xxh3_128('hello')) AS large;
-- UBIGINT, UHUGEINT
```

```
INSTALL hashfuncs FROM community; LOAD hashfuncs
```

Inflector



Normalize names, keys, and query result columns.

Mixed names → Choose a format → Consistent shape

Apply consistent naming conventions to strings, struct keys, or every column of a query. Combine case conversion with English inflection helpers when cleaning imported schemas, generating identifiers, or shaping data for another system.

Derive table and word names

```
SELECT inflector_to_table_case('UserAccount');
-- user_accounts
SELECT inflector_to_plural('person');
-- people
SELECT inflector_to_singular('people');
-- person
```

```
INSTALL inflector FROM community; LOAD inflector
```

JSONata

Query and reshape JSON with compact expressions.

JSON document → JSONata expression → New JSON shape

Filter nested arrays, construct output objects, and aggregate values within a JSON document. JSONata keeps these transformations in one SQL expression and returns JSON that can be cast, extracted, or expanded by DuckDB.

Filter with a bound threshold

```
SELECT jsonata(
  '[items[price > $limit].id]',
  '{"items":[{"id":"A","price":20},' ||
  '{"id":"B","price":120}]}',
  '{"limit":100}'
);
-- ["B"]
```

```
INSTALL jsonata FROM community; LOAD jsonata
```

MiniJinja



Render Jinja-style templates directly from SQL.

Template → JSON context → Rendered text

Turn JSON context into rendered text using familiar Jinja variables, filters, loops, and conditions. Generate one HTML fragment, message, or configuration block per row while keeping the source data and rendering query together.

Loop through a list

```
SELECT minijinja_render_with_context(
  '{% for x in xs %}{{ x }}' ||
  '{% if not loop.last %}, {% endif %}' ||
  '{% endfor %}',
  '{"xs":["A","B","C"]}'
);
-- A, B, C
```

```
INSTALL minijinja FROM community; LOAD minijinja
```

QuickJS

Bring JavaScript expressions into DuckDB queries.

SQL arguments → JavaScript → Text / JSON / rows

Evaluate a JavaScript expression, call a function with SQL arguments, or expand an array into rows. Use the embedded QuickJS engine for transformations that benefit from JavaScript control flow, string methods, or JSON reshaping.

Unpack a transformed array

```
SELECT result::INTEGER AS value
FROM quickjs(
  'parsed_arg0.map(x => x * arg1)',
  '[1,2,3]', 3
);
-- 3, 6, 9
```

```
INSTALL quickjs FROM community; LOAD quickjs
```

RapidFuzz



Fuzzy string comparisons beside your data.

Normalize text → Score candidates → Rank matches

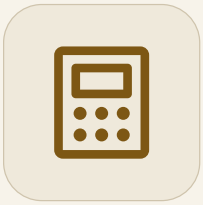
Score candidate pairs using edit-distance and token-based matching. Normalize text first, select a score that matches your data, then rank or threshold the candidates.

Find the closest candidate

```
SELECT name,
  rapidfuzz_ratio('query farm', lower(name))
    AS score
FROM (VALUES ('Query Farm'), ('Query Firm'),
             ('Quiet Barn')) AS t(name)
ORDER BY score DESC
LIMIT 2;
```

```
INSTALL rapidfuzz FROM community; LOAD rapidfuzz
```

Rhai



Evaluate expressions and rules stored as text.

Script text → Row context → JSON result

Run Rhai expressions inside a query, pass row values as JSON context, and read a typed success or error result. Useful for configurable business rules, small transformations, and scripts that need local variables or branching.

Apply one rule to each row

```
SELECT salary, evalexpr_rhai(
  'context.salary >= 100000',
  {'salary': salary}
).ok::BOOLEAN AS eligible
FROM (VALUES (80000), (120000)) t(salary);
-- false, true
```

```
INSTALL evalexpr_rhai FROM community; LOAD evalexpr_rhai
```

Tera



Render templates from inline text or named files.

Text or file → JSON context → Rendered text

Render JSON context with Tera's variables, filters, loops, and conditionals. Keep short templates in SQL, or load named files for larger templates and inheritance. Results are ordinary text columns ready for export or further processing.

Render a plain-text list

```
SELECT tera_render(
  '{% for x in xs %}{{ x }}' ||
  '{% if not loop.last %}, {% endif %}' ||
  '{% endfor %}',
  '{"xs":["B&O","Acme"]}',
  autoescape := false
);
-- B&O, Acme
```

```
INSTALL tera FROM community; LOAD tera
```

TSID



Generate sortable IDs and recover their timestamps.

Generate ID → Store as VARCHAR → Sort or decode

Create time-prefixed identifiers as 32-character hexadecimal strings and recover their embedded timestamps. Use them as internal keys, recent-first sort values, or pagination cursors when generation time is useful alongside a compact textual identifier.

Page through recent IDs

```
SELECT * FROM events
ORDER BY id DESC LIMIT 2;

SET VARIABLE cursor = (
  SELECT id FROM events
  ORDER BY id DESC LIMIT 1 OFFSET 1
);
SELECT * FROM events
WHERE id < getvariable('cursor')
ORDER BY id DESC LIMIT 2;
```

```
INSTALL tsid FROM community; LOAD tsid
```

A5



Equal-area cells for geographic data.

Longitude / latitude → A5 cell ID → Group / join / roll up

Turn longitude and latitude into pentagonal cell IDs for grouping, joining, and geographic rollups. A5 uses equal-area cells at each resolution, with integer IDs and functions for hierarchy, neighborhoods, and geometry conversion.

Count observations by cell

```
WITH points(lon,lat) AS (VALUES
  (-74.0060,40.7128), (-74.0061,40.7129),
  (-73.9857,40.7484)
)
SELECT a5_lonlat_to_cell(lon,lat,15) AS cell,
       count(*) AS n
FROM points
GROUP BY cell;
```

```
INSTALL a5 FROM community; LOAD a5
```

Apache DataSketches



Approximate answers from compact aggregate state.

Rows → Sketch state → Merge / estimate

Build sketches over rows, persist their typed state, and query estimates later. Select a family for distinct counts, quantiles or frequent items; merge states to summarize partitions.

Estimate p50 and p95

```
WITH agg AS (
  SELECT datasketch_kll(200, i::DOUBLE) AS s
  FROM range(10000) AS t(i)
)
SELECT datasketch_kll_quantile(s, 0.50, true)
         AS p50,
       datasketch_kll_quantile(s, 0.95, true)
         AS p95
FROM agg;
```

```
INSTALL datasketches FROM community; LOAD datasketches
```

Stochastic



Probability distributions as SQL functions.

Distributions → PDF · CDF · quantile → Sampling

Densities, cumulative probabilities, quantiles and random samples for common continuous and discrete distributions, callable from any query.

Estimate π with a Monte Carlo query

```
WITH draws AS MATERIALIZED (
  SELECT
    dist_uniform_real_sample(-1, 1) AS x,
    dist_uniform_real_sample(-1, 1) AS y
  FROM range(100000)
)
SELECT 4.0 * AVG(
  (x*x + y*y ⩽ 1)::INTEGER
) AS pi_estimate
FROM draws;
```

```
INSTALL stochastic FROM community; LOAD stochastic
```

Bitfilters



Compact membership checks for large sets.

Hash keys → Build filter → Probe & verify

Aggregate hashed keys into a compact BLOB, then probe it to reject definite nonmembers. Use an exact check for candidates that survive the filter.

Confirm the match

```
SELECT i
FROM known_filter, range(995, 1005) AS t(i)
WHERE xor8_filter_contains(f, hash(i))
  AND i IN (SELECT range FROM range(1000));
```

```
INSTALL bitfilters FROM community; LOAD bitfilters
```

GeoSilo



Compact geometry with explicit precision.

Native GEOMETRY → Scale + delta encode → Store / inspect / decode

Store geometries as scaled integer coordinates with delta encoding. GeoSilo provides a GEOSILO column type, direct measurements and bounds, and explicit encode/decode functions for BLOB pipelines. Choose precision before comparing storage size or query performance.

Store a projected polygon

```
CREATE TABLE plots (
  geom GEOSILO('EPSG:3857')
    USING COMPRESSION zstd
);
INSERT INTO plots VALUES (
  'POLYGON((0 0,10 0,10 10,0 10,0 0))'
    ::GEOMETRY
);
```

```
INSTALL geosilo FROM community; LOAD geosilo
```

Lindel



Space-filling curves for faster range filters.

Multi-column keys → Hilbert order → Pruned Parquet

Hilbert and Morton encodings turn several columns into one sortable key. Ordering Parquet by that key keeps nearby points in the same row groups, so range filters skip more of the file.

Cluster a Parquet file on two columns

```
COPY (
  SELECT x, y
  FROM (VALUES (3, 5), (1, 1), (7, 0),
               (2, 6), (5, 3), (0, 4))
       AS points(x, y)
  ORDER BY
    hilbert_encode([x, y]::UINTEGER[2])
) TO 'points.parquet' (FORMAT PARQUET);
```

```
INSTALL lindel FROM community; LOAD lindel
```

Marisa



Static dictionaries with fast prefix lookup.

String column → Trie BLOB → Lookup / prefix

Build a compact MARISA trie from a column of strings. Store the trie as a BLOB and use it for exact membership, autocomplete, or prefix-rule matching.

Match stored prefixes

```
WITH rules AS (
  SELECT marisa_trie(s) AS trie
  FROM (VALUES ('U'), ('US'), ('USA')) t(s)
)
SELECT marisa_common_prefix(trie, 'USA', 10)
FROM rules;
```

```
INSTALL marisa FROM community; LOAD marisa
```

JSON Schema



Validate JSON contracts and apply declared defaults.

Schema + JSON → Validate → Patch or enrich

Check JSON documents against a schema inside SQL, then fill missing properties from schema defaults. Useful at ingest and during audits. Invalid documents raise errors in this build, so choose strict validation or a deliberate TRY wrapper.

Flag valid and invalid rows

```
SELECT payload, coalesce(try(
  json_schema_validate(
    '{"type":"integer"}', payload
  )), false) AS valid
FROM (VALUES ('5'::JSON), ('"five"'::JSON))
  t(payload);
-- 5: true; "five": false
```

```
INSTALL json_schema FROM community; LOAD json_schema
```

Cronjob



Schedule recurring SQL in a live DuckDB process.

SQL + schedule → Live scheduler → Inspect / cancel

Register SQL with a six-field cron expression, inspect jobs and cancel by ID. The scheduler lives inside the DuckDB process, making it useful for short-lived local automation.

Register a job

```
CREATE TEMP TABLE scheduled AS
SELECT cron(
  'SELECT 42', '0 */5 * * * *'
) AS job_id;
```

```
INSTALL cronjob FROM community; LOAD cronjob
```

Events



Query and transaction events as a JSON stream.

DuckDB event → JSON line → Handler stdin

Forward DuckDB lifecycle events to an executable handler as JSON lines on standard input. Use the stream for operational logs, query timing and transaction traces, with a session tag to identify each running process.

Capture query activity

REQUIRES SETUP

```
SET events_session_name = 'analytics';
SET events_types = [
  'query_begin', 'query_end'
];
SET events_async = TRUE;
SET events_destination =
  '/usr/local/bin/query-events';
SELECT sum(i) FROM range(1000) t(i);
```

```
INSTALL events FROM community; LOAD events
```

Fuzzycomplete

Aa

Fuzzy SQL completion with a catalog-aware matcher.

Partial SQL → Catalog + matcher → Suggestions

Suggest SQL keywords and catalog names from a partial query using a fuzzy matcher. Fixed v1.5 builds activate even when the DuckDB CLI has already loaded stock autocomplete; inspect suggestions through the table function.

Try a catalog abbreviation

REQUIRES SETUP

```
CREATE TEMP TABLE getNotificationStatus(
  id INTEGER
);
-- In a compatible interactive CLI, type:
-- SELECT * FROM gnst
-- Then press Tab before submitting.
```

```
INSTALL fuzzycomplete FROM community; LOAD fuzzycomplete
```

Textplot



Small charts inside your query results.

Numbers → Plot function → VARCHAR chart

Render a numeric sequence or progress value as terminal-friendly text. Keep the underlying values for sorting and computation; add plots as a compact visual summary.

Aggregate in order

```
SELECT tp_sparkline(
  array_agg(value ORDER BY step),
  theme := 'ascii_basic'
) AS trend
FROM (VALUES (1, 4.0), (2, 7.0),
             (3, 5.0)) AS t(step, value);
```

```
INSTALL textplot FROM community; LOAD textplot
```

WebMacro



Share SQL macros through a trusted URL.

SQL at a URL → Load definition → Call the macro

Load a DuckDB macro definition from an HTTP or HTTPS endpoint and call it in ordinary SQL. Use reviewed, versioned SQL files to distribute reusable scalar expressions and table queries across notebooks and applications.

Inspect and remove a macro

```
SELECT function_name, macro_definition
FROM duckdb_functions()
WHERE function_type = 'macro'
  AND function_name = 'normalize_tag';
DROP MACRO normalize_tag;
```

```
INSTALL webmacro FROM community; LOAD webmacro
```

Query.Farm

KEEP THE FULL SET

Printable reference sheets for every extension.

Function signatures, setup notes and tested examples. Each extension page also runs its examples live in your browser.

query.farm/products/extensions

Open source · Installs with one line from the DuckDB community repository